Inventory Management Decisions (Product Perspective)

Discussions related to exploring Inventory management as a separate entity, not dependent on syncing Item QoH with QBO

Should Method or QBO be the source of truth (SOT) for IMS?

🎯 **TLDR:** Method should be the source of truth for inventory because QBO is built for accounting, not inventory. It supports limited inventory lingo and lacks core workflows (multi-warehouse, bins, stock reservations, etc) that our ICP requires. Method is introducing these native capabilities to close the parity gap with QBDT and meet customer demand, while QBO remains the SOT for financials.

Decision	Why?	Advantages	Disadvantages
Method should be the SOT for inventory.	1. QBO does not support real inventory workflows. 2. Method is introducing an entire suite of native inventory management features. 3. Our target ICP needs multi-warehouse, bins, stock reservations, etc. — things QBO simply doesn't do. 4. QBO = accounting system, not inventory platform. Businesses already use Katana/SOS for inventory because QBO is insufficient.	1. Removes dependency on QBO's limited inventory model (single QOH bucket). 2. Positions Method as a true IMS (aligns with our product vision of delivering a native inventory management solution in house). 3. Reduces churn risk and satisfies are potential early adopters (QBDT users migrating away need parity features).	1. Engineering lift: Method must calculate & maintain inventory state with high accuracy 2. Puts more pressure on Method to deliver reconciliation & trust tools (ledger, adjustments). 3. Migration risk for existing customers relying on QBO's QOH.

	5. Our customers have complained of the lack of inventory parity between QBO and QBDT, requesting Method to deliver their desired solution.		

If Method is the SOT, do we decouple QOH from QBO?

🎯 **TLDR:** We should decouple QOH from QBO because QBO's model can't support real inventory workflows. Method needs to own QOH to maintain inventory integrity, deliver the IMS features customers are asking for, and build trust. QBO QOH can still be logged for audit, but never overwrite Method's values.

Decision	Why?	Advantages	Disadvantages
Decoupled item QOH that syncs with QBO and item total QOH.	1. User trust principle: Inventory integrity = trust. Users need confidence their IMS numbers won't get overwritten by QBO's simplified data. 2. Future-proof for integrations: Katana already ignores QBO QOH. Decoupling now avoids messy 3-way conflicts when Method integrates with external IMSs.	1. Eliminates risk of QBO overwriting Method's QOH. 2. Keeps Method's IMS consistent and independent. 3. Prevents QBO sync complexities. 4. Enforces true IMS ownership resulting in increased trust & integrity.	1. Users may expect QBO & Method totals to always match → education required. 2. Some accountants may push back if they prefer QBO as the "master" dataset.

When Katana is connected, does it become the new SOT?

🎯 **TLDR:** When integrated, customers expect Katana to be the source of truth for inventory. Katana is built as a full-scale IMS, while Method complements it by handling front-office sales operations and QBO manages accounting.

Decision	Why?	Advantages	Disadvantages
Katana becomes the SOT for inventory when connected.	1. Katana is a dedicated IMS with a full scale inventory solution. 2. Method's role = front office / sales operations UI (sales orders, invoices, purchasing) when integrated. 3. Katana already integrates with QBO for accounting. Forcing Method to override Katana would create conflicts and frustration.	1. Clear role separation: Katana = warehouse ops, Method = sales ops, QBO = accounting. 2. Reduces complexity of a 3-way sync 3. Aligns with user expectations: Customers expect Katana to be the SOT if they are paying for a dedicated IMS.	1. Method must defer authority to Katana while still surfacing data. 2. Requires careful comms to users ("your inventory is controlled by Katana, not Method"). 3. Risk of product confusion if customers expect Method IMS features to "win" over Katana.

What happens when conflicting data flows in from Katana and QBO?

🎯 **TLDR:** Katana should always be prioritized as the source of truth for inventory, since customers trust it as their dedicated IMS, while QBO remains focused on accounting. This clear hierarchy avoids ambiguity, protects inventory integrity, and keeps the Method–Katana integration simple and reliable.

Decision	Why?	Advantages	Disadvantages

Prioritize Katana at all times.	1. Users expect Katanas value to “win” since it’s a dedicated IMS and are deeply invested into their platform. 2. QBO adjustments are often accounting-driven. These should not overwrite physical inventory states. 3. We should align our language with Katana, to increase trust and integrity.	1. Clear hierarchy avoids ambiguity: Katana always wins for inventory. 2. Matches real-world usage: customers use Katana for inventory, QBO for accounting. 3. Keeps Katana + Method integration “happy path” simple and reliable.	1. Tightly couples Method to Katana’s model — risk if Katana changes APIs/flows. 2. Users may be surprised if QBO values “don’t matter” anymore.

Discussion related to SO/PO fulfillment flow

How much do we want to mimic Katana? At what point do we start becoming a competitor vs a partner?

🎯 **TLDR:** We’re not here to copy Katana — we’re here to complement it. Method delivers the core inventory foundation every SMB expects, while Katana extends into advanced workflows. That balance is what makes our hybrid strategy powerful — CRM-first, integration-friendly, and built for growth.

Response

Our goal was never to replicate Katana, but to enable scalability for our customers through a hybrid approach — one that balances native capability and integration flexibility.

Our Hybrid Strategy clearly defines that boundary:

> **Method provides the foundational inventory layer natively**, while **Katana extends it for advanced operations**.
> This ensures Method remains a CRM-first platform that empowers small-to-mid-sized businesses to scale into more complex inventory environments without ever needing to churn and start over.

The Strategic Balance

By design, Method covers the "core" inventory requirements all SMBs expect to work without customization, while Katana handles depth and specialization.

Why This Isn't "Mimicking Katana"

1. **We're solving for the baseline expectation, not parity.**
   Every IMS — from SOS, Katana, Cin7, Zoho Inventory — supports these core features in a similar fashion. Delivering those core capabilities in Method is an operational requirement, not imitation, and one that our customers expect to function OOB.
2. **We're reinforcing our ecosystem, not fragmenting it.**
   Integration with Katana extends our reach into complex inventory workflows that Method doesn't intend to own.
3. **We're enabling user progression.**
   Customers can start and grow within Method's native system, then graduate seamlessly to Katana when their operational complexity increases — rather than churn to another platform.
4. **We're distinct in identity.**
   - Katana is a distinguished Inventory Management System.
   - Method is a CRM that optionally includes core inventory to unify sales, service, and stock visibility.

   This differentiation allows us to complement Katana, not compete with it.

When We'd Risk Becoming a Competitor

We only cross into competitive territory if we:

- Attempt to replicate MWD-grade features (e.g., BOM routing, shop-floor control, production costing).
- Build full ERP capabilities beyond CRM-centric inventory visibility.

- Stop promoting or integrating with Katana for advanced workflows.

As long as we continue delivering:

- Core inventory functions natively (what every SMB expects), and
- Deep integration pathways into Katana for advanced needs,

—we remain partners serving different maturity stages of the same customer journey.

The Takeaway

> We're not mimicking Katana — we're building the on-ramp to Katana. Method's hybrid model ensures we serve customers at every operational stage:
> from CRM-centric sales visibility to fully automated production workflows — together, not in competition.

Which fulfillment version should we build for MVP — V1 (partial-per-line delivery) or V2 (whole-line only)?

🎯 **TLDR:** Modern inventory-enabled systems are expected to allow partial delivery of line items. Forcing full-line delivery would break standard fulfillment workflows, distort reporting, and reduce usability for many inventory focused customers.

Response

Product Expectation

Users should be able to partially deliver/receive individual line items rather than being forced to deliver/receive the full ordered quantity per line.

Product Rationale

1. **Reflects real-world workflows:**

- In most real-world scenarios, businesses don't have all ordered quantities available at once. Allowing partial delivery per line item lets them ship what's in stock now while keeping the remaining quantities committed for future fulfillment. The same can be applied to receiving products from vendors.
- Requiring full-line delivery creates unnecessary roadblocks — it forces users to either delay shipments, duplicate orders, or resort to manual workarounds. This not only adds friction but also breaks trust in the system's flexibility and how real stock movements flow.

- By supporting partial-per-line delivery, Method aligns with how fulfillment actually works, reducing administrative overhead and improving the overall user experience.

2. **Preserves data integrity:**
   Each fulfilment updates inventory precisely. This ensures the system's stock data always mirrors physical reality.
3. **Builds user trust:**
   Users can rely on Method to represent their true fulfillment status at any moment. Blocking partial deliveries would cause discrepancies between the warehouse and the system, leading to workarounds and erosion of trust.
4. **Aligns with industry standards:**
   All modern inventory platforms (Katana, SOS, Cin7, Zoho, Fishbowl, QuickBooks Advanced Inventory) support partial-per-line fulfillment. It's expected behaviour and table stake in modern inventory management solutions.

From a product standpoint, the expected behaviour is to allow partial deliveries/receiving per line item within a SO/PO. This approach maintains inventory integrity, strengthens user trust, and aligns Method with standard fulfillment workflows across the inventory management industry.

Should inventory and accounting flows be kept fully separate, or should we tie delivery/receiving to invoicing/billing events?

🎯 **TLDR:** For our ICP, inventory and accounting should stay separate but synchronized - delivery and receiving should drive stock updates, while invoicing and billing follow fulfillment. This ensures operational trust, financial accuracy, and scalability within our hybrid inventory model.

Response

1. **Our ICP Is Physically Grounded**

Method's target users — small (1–100 employee) B2B distribution companies — manage *real stock in real locations*.
Their workflows revolve around physical inventory truth:

- Stock is committed when a Sales Order is created.
- Stock moves when goods are delivered or received.
- Revenue or expense is recognized once the goods change hands.

For them, **tying invoicing directly to stock movement** would blur the operational and financial timelines — something even QuickBooks Desktop and Katana keep decoupled.

They expect each process to be **accurate on its own**, but **linked for traceability**.

2. Operational Separation Builds Trust

Inventory management is about *physical accuracy*.
Accounting is about *financial accuracy*.
When one is forced to trigger the other, you risk mistrust on both sides.

By keeping them separate:

- Deliveries and Receipts can occur on time, even if billing is delayed.
- Invoices and Bills can be issued later, once fulfillment is confirmed.
- The system can still cross-link both for audit and reconciliation.

This ensures **Quantity on Hand, Committed, Expected, and Available** stay true.

3. Why This Matters for MWD SMBs

Your ICP values:

- **Inventory accuracy** over accounting flexibility
- **Operational clarity** over financial shortcuts
- **Ease of correction** (undo delivery, fix quantity, re-deliver)

If invoicing and delivery were tightly coupled, a simple mistake (like delivering to the wrong warehouse) could require voiding financial documents — creating friction, not trust.

By separating them, Method allows **operational corrections without accounting disruption** — exactly what growing SMBs need.

Product Principle

> Keep inventory and accounting **separate in logic**, **linked in traceability**, and **sequenced in workflow**. Delivery/Receiving events update inventory first. Invoicing/Billing events follow those physical movements to record revenue or cost.

Should sales orders and purchase orders be editable after creation?

🎯 **TLDR:** Sales Orders and Purchase Orders should be editable after creation, but with conditional restrictions. Users should have the ability to freely modify open orders before fulfillment, while partially or fully fulfilled orders should only allow edits to remaining unfulfilled or non-stock-impacting fields.

Response

Sales Orders and Purchase Orders should remain editable after creation — up until the point where fulfillment or financial transactions occur (delivery/receiving/invoice/bill). Once such downstream events exist, edits must follow validation rules or be restricted to non-impacting fields.

Why Edits Must Be Allowed (Initially)

1. **Operational Reality**
   - Orders often evolve — customers adjust quantities, swap items, or change delivery locations before fulfillment.
   - Vendors may update pricing or confirm only partial supply before receipt.
   - Preventing edits too early forces users to delete and recreate orders, which fragments history and breaks audit trails.
2. **Efficiency & UX**
   - Editable orders reduce friction for front-office users.
   - Sales reps and purchasing teams need flexibility before fulfillment begins.
3. **Industry Standard**
   - All modern IMS platforms (Katana, Zoho) allow edits until fulfillment starts.

Integrity & Trust Angle

Editable SOs/POs increase usability but must preserve transactional truth:

- The system should never retroactively alter committed, expected, or on-hand quantities from already fulfilled lines.
- This transparency maintains user trust in inventory accuracy.

This approach balances flexibility, integrity, and trust, aligning with standard best practices across modern inventory systems.

If the PO originated from QBO and synced to Method, any changes made to the PO in QBO will still reflect in Method.

State	Inventory Impact	Accounting Impact	Line Items Editable?	Line Items Deletable?	Expected Behaviour & Rationale
Open (Committed / Expected)	❌	❌	✅ **Yes** — All unfulfilled line items fully editable	✅ **Yes** — All unfulfilled line items can be deleted	Deleting or editing adjusts committed/expected stock accordingly. Safe to modify since no physical or financial movement has occurred.
Partially Fulfilled (Some Delivered/Received)	✅	❌	⚠️ **Limited** — Only *unfulfilled* line items are editable; fulfilled ones are read-only	⚠️ **Limited** — Only unfulfilled line items can be deleted; fulfilled ones can only be reversed via Undo action	Delivered/received lines are locked for edit/delete to preserve inventory history. System auto-updates committed/expected balances when open lines are modified or removed.
Fully Fulfilled (All Delivered/Received, Not	✅	❌	❌ **No** — Fulfilled lines are read-only	❌ **No** — Must use Undo action to reverse	Editing or deleting would rewrite stock history.

Yet Invoiced/Billed)					Only structured undo actions allowed before invoicing or billing.
Delivered/Received & Invoiced/Billed	✅	✅	❌ **No** — Fulfilled lines are read-only	❌ **No** — Undo action disabled; reversal must occur by deleting the linked invoice/bill first before proceeding	Once invoiced/billed, the transaction is financially locked. To reverse, users must first delete the associated invoice/bill to perform an undo action. This ensures stock movements stay synchronized with accounting records and prevents desynchronization between inventory and financial systems

Should we allow undoing delivered/received items in SO/PO?

🎯 **TLDR:** Undoing delivered or received items is a necessary safeguard against real-world mistakes. A well-designed undo flow lets SMBs correct fulfillment errors safely, restore inventory trust, and preserve historical accuracy — without making them start over.

Response

Conceptual Framing

- Delivered (SO) and Received (PO) states indicate physical movement of stock has already occurred.
- Undoing those events means reversing inventory balances (Quantity on Hand, Committed, Expected, Available), which is inherently risky unless carefully tracked.
- However, businesses sometimes need to reverse a fulfillment (e.g., wrong delivery, damaged goods, supplier returns).

When It's Needed

Undo actions are essential because:

- Users make mistakes (wrong location, wrong quantity, wrong item).
- Orders may be canceled after shipment or supplier returns may occur.
- You need to restore stock and trust without breaking transaction history.

If Undoing is Not Allowed

1. **Inventory Becomes Rigid — Errors Require Workarounds**

- Once an item is marked *Delivered* or *Received*, users can't fix mistakes like:
  - Wrong quantities delivered or received
  - Wrong warehouse or location selected
  - Delivery recorded before actual shipment
- To fix these, users must:
  - Create *new offsetting orders* (e.g., a "Return" Sales Order or negative PO)
  - Or manually create **Stock Adjustments** to fix balances

Result: Inventory balances stay correct *only if users know how to reverse manually*. That increases the cognitive load and support volume.

SMBs make real-world mistakes — a good system should let them fix those mistakes safely, not make them start over.